

COMPUTER VISION

2025 - 2026

>02. CAMERA GEOMETRY & IMAGE FORMATION

UTRECHT UNIVERSITY

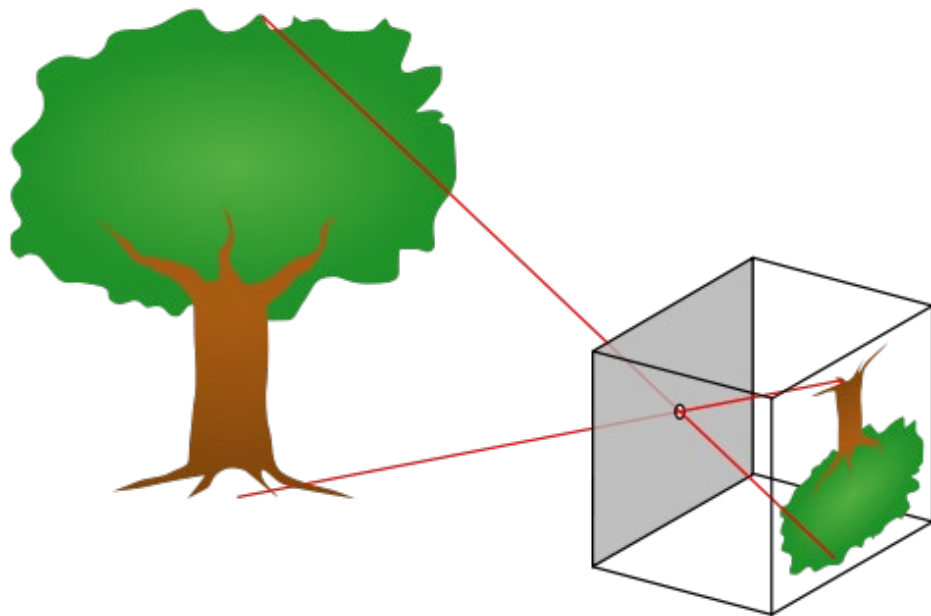
RONALD POPPE

CAMERAS

2D projection of 3D world onto an image:

- “shape”: geometry
- “color”: radiometry

Assignment walkthrough



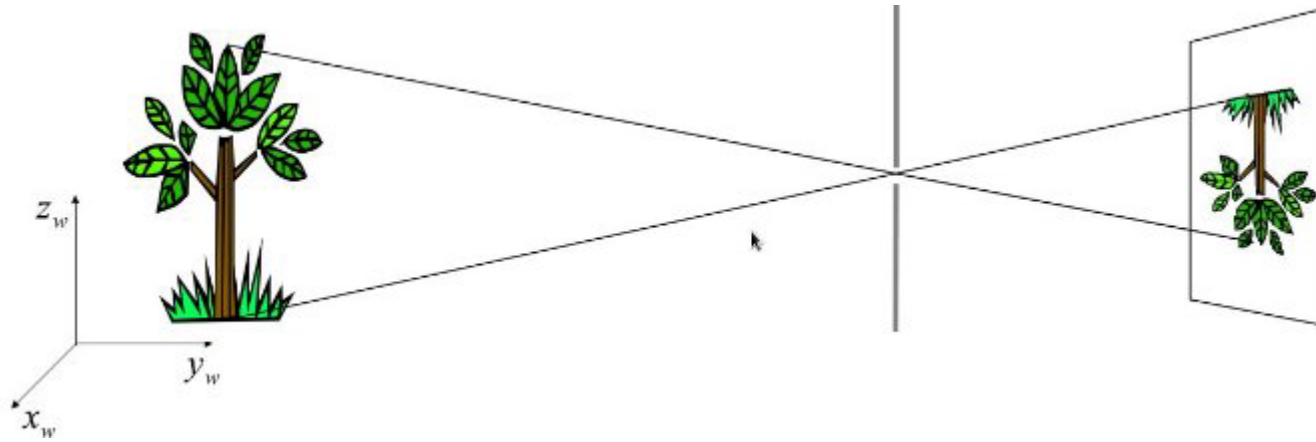
CAMERA GEOMETRY

CONCEPT

We have coordinate systems:

- 3D world coordinates (in meters)
- 2D image coordinates (in pixels)

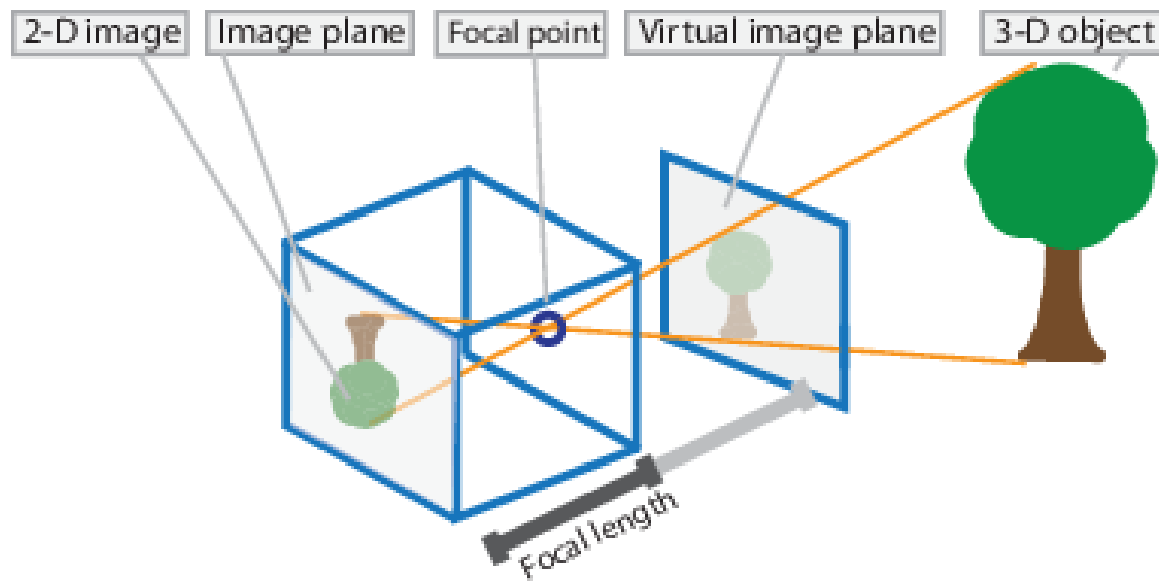
How to model the projection from 3D to 2D?



PINHOLE CAMERA MODEL

Often, we assume a pinhole camera model

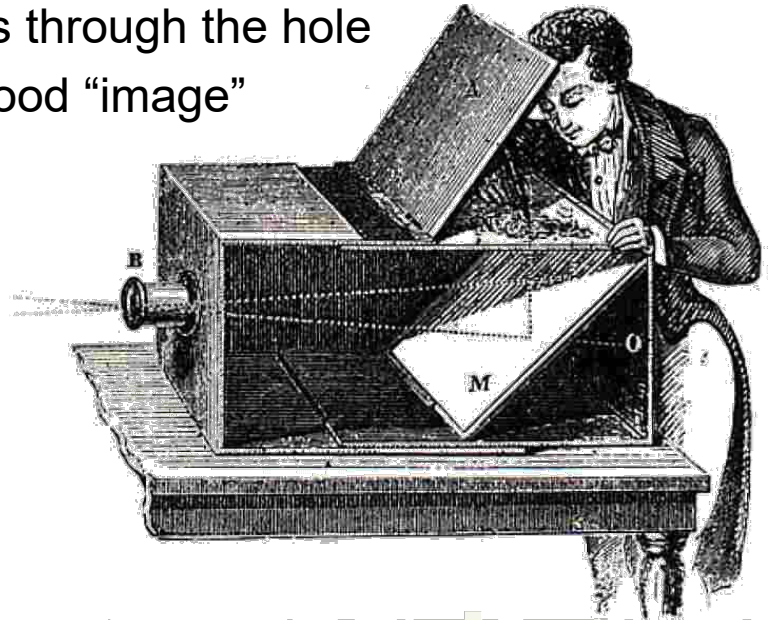
- Think of light as rays
- The “lens” is a point



PINHOLE CAMERA MODEL²

Camera obscura can be considered a pinhole camera

- Light-receptive paper used to “record” image
- Drawback: limited amount of light passes through the hole
- Long exposure times are needed for a good “image”



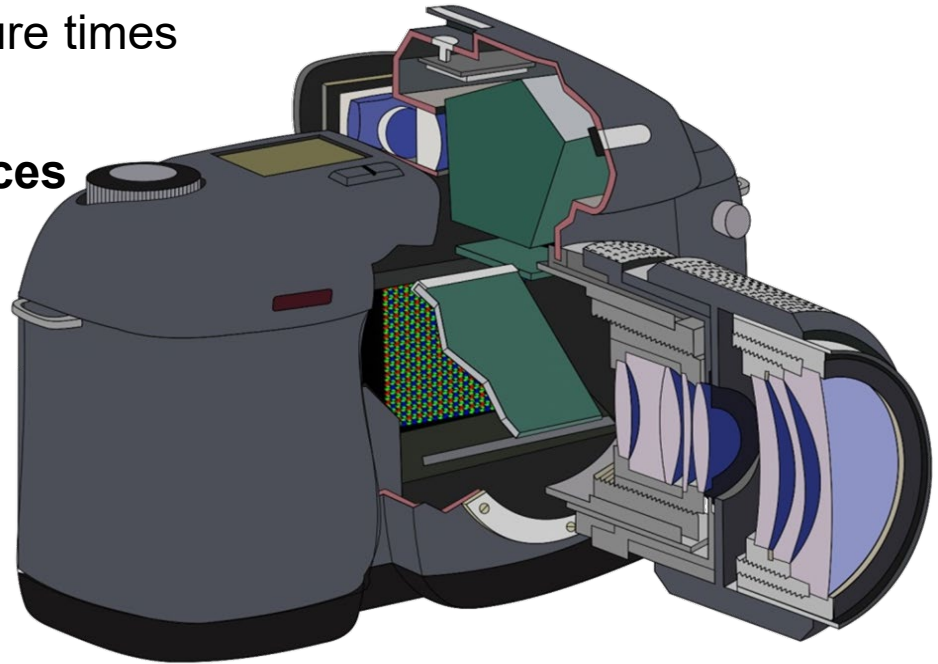
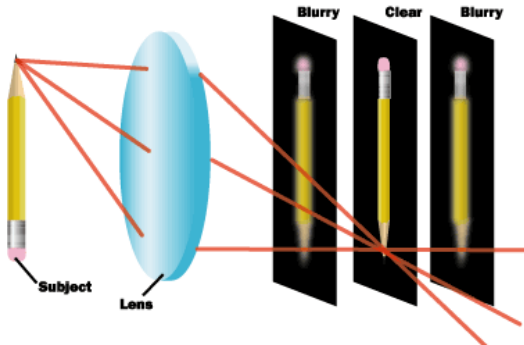
DIGITAL CAMERAS

Modern (digital) cameras use lenses to focus light

- Larger lens area → shorter exposure times

Zooming for focus at varying distances

- “Moves the virtual image”
- Causes blur at other depth ranges



FROM 3D TO 2D

How a 3D object appears on the 2D image depends on:

- Properties of the lens
- Object's relative position to the lens

In an image projection, the depth information is lost

Projection can be modeled as:

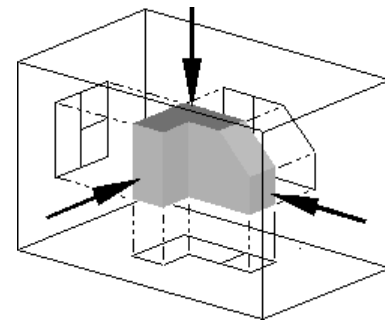
- Orthographic projection
- Perspective projection

ORTHOGRAPHIC PROJECTION

Drop the depth information (z-dimension):

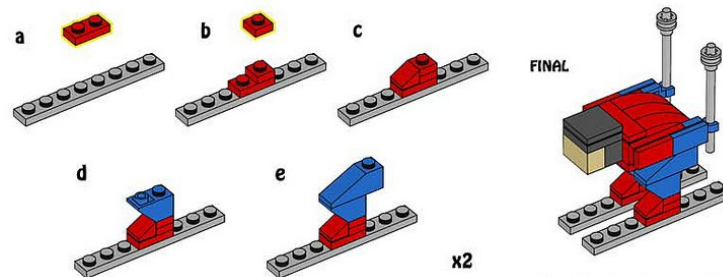
$$x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} p$$

With x a 2D coordinate and p a 3D point.



Not consistent with how we see the world

- Objects further away do not appear smaller

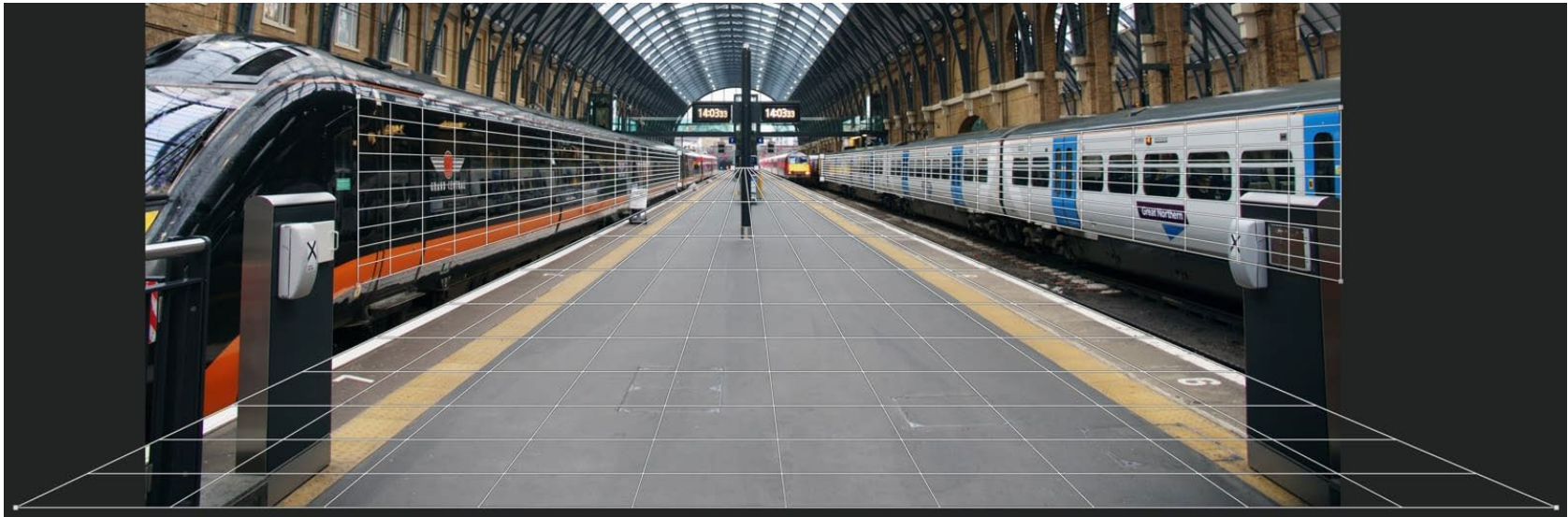




PERSPECTIVE PROJECTION

Reality: objects further away appear smaller

- Depth affects projection
- Straight lines remain straight after projection



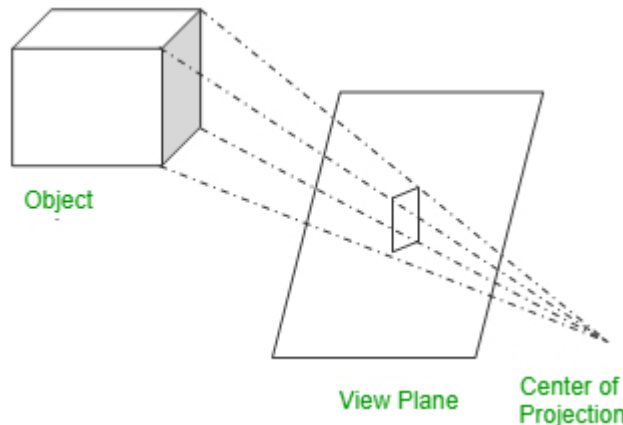
PROJECTING POINTS

We look at how 3D structures are projected onto a 2D image

- We regard structures as a series of 3D points
- We would like to know the corresponding 2D image locations

Perspective projection is assumed

- All 3D points converge in a single point (lens)
- Virtual image plane is used to model the image



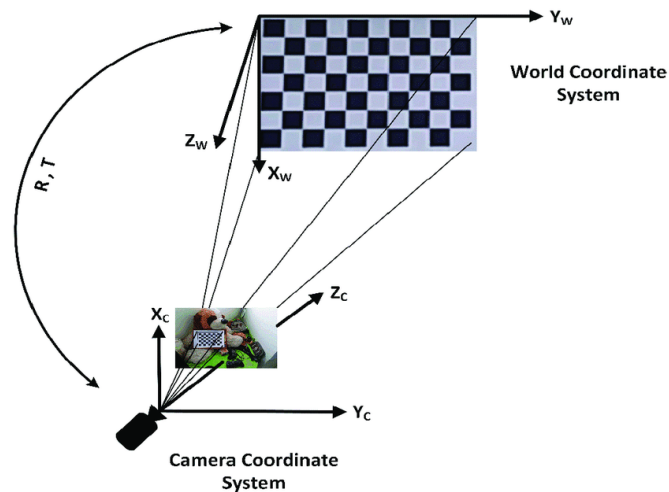
CAMERA MATRIX

Camera matrix P represents the projection from 3D world to 2D image

- Images have a 2D local axis system (in pixels)
- World has arbitrary 3D origin and 3D axis system (in meters)

We use intermediary 3D camera coordinates

- Translation and rotation of the camera
- World points expressed relative to the camera



CAMERA MATRIX²

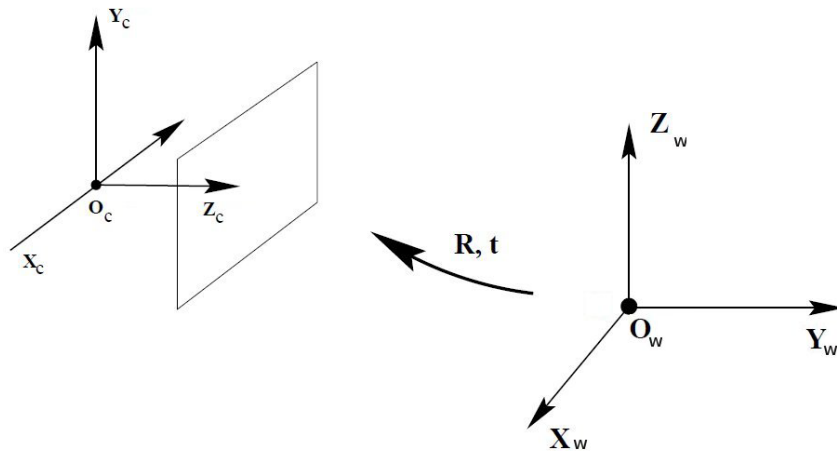
Camera matrix P contains extrinsic and intrinsic parameters

- Extrinsic: convert 3D world to 3D camera coordinates
- Intrinsic: convert 3D camera to 2D image coordinates

EXTRINSIC PARAMETERS

Express 3D world coordinates in 3D camera coordinates

- Account for the position and orientation of the camera in the world
- Recall a 3D rotation: $\bar{x}' = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \bar{x}$ (with $\bar{x}$ the augmented vector)
- We have to determine extrinsic parameters $\mathbf{R}$ (3x3) and $\mathbf{t}$ (3x1)



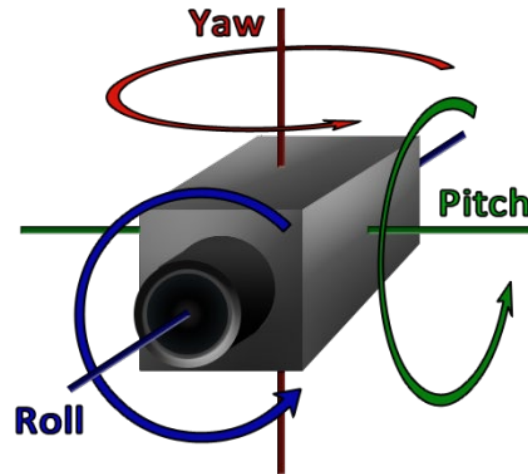
EXTRINSIC PARAMETERS²

What we do is:

- Find the difference in orientation between camera axis system and world axis system: R
- Find the location of world origin O seen from camera center C : t

Referred to as a 6DOF problem:

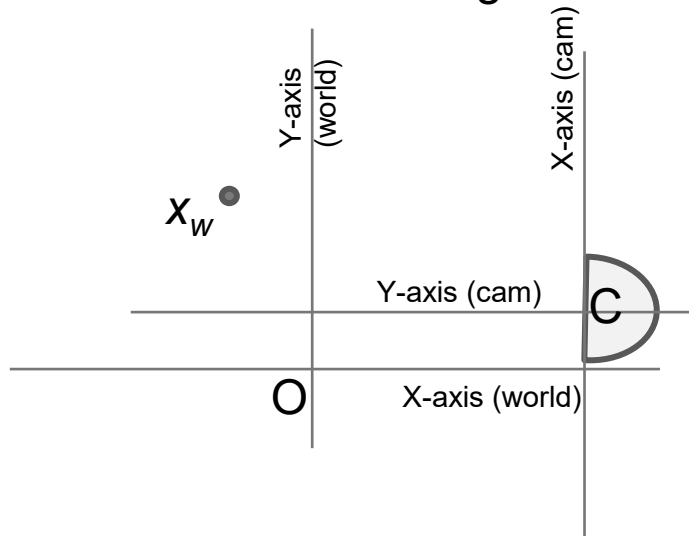
- Three degrees of freedom in translation
- Three degrees of freedom in rotation (yaw, pitch, roll)



EXTRINSIC PARAMETERS³

Example in 2D:

- For convenience: camera **C** direction is orthogonal to world
- We “look” along the camera’s y-axis (usually z-axis in 3D)



x_w is our world coordinate (-1, 3)

Our camera center **C** is at (4, 1)

Our camera is rotated 90 degrees counter clockwise:

- $\cos(-90) = 0$, $\sin(-90) = -1$
- So $R = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$

Origin **O** expressed in **C**'s axis system: (-1, 4):

$$\overline{x_c} = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \overline{x_w} = \begin{bmatrix} 0 & 1 & -1 \\ -1 & 0 & 4 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -1 \\ 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 5 \\ 1 \end{bmatrix}$$

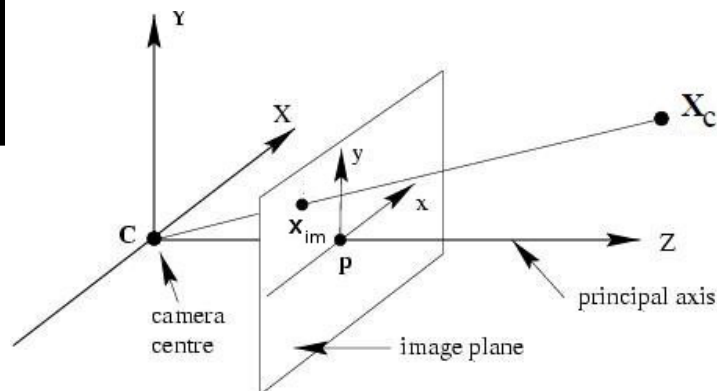
INTRINSIC PARAMETERS

We now project 3D camera coordinate $\mathbf{x}_c$ on the 2D image plane: $\mathbf{x}_{im}$

- $\mathbf{x}_c$ is in camera coordinates, with origin $\mathbf{C}$ and principal axis Z

Matrix K represents the projective transform:

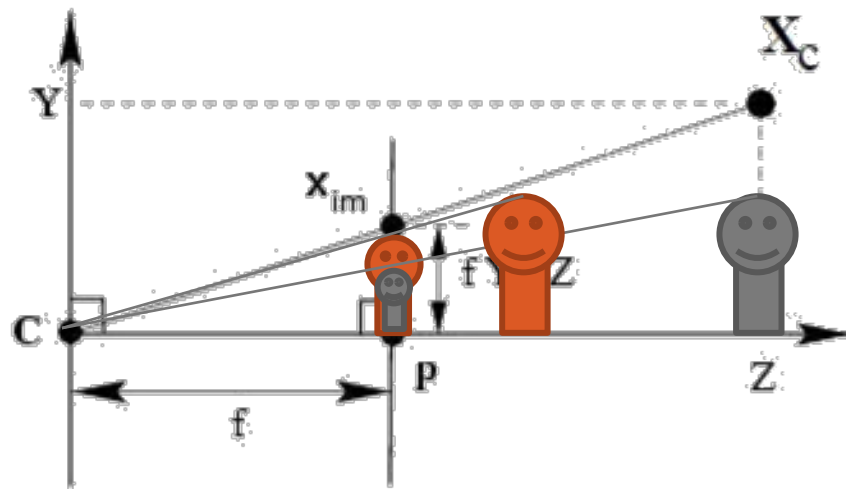
$$\bar{\mathbf{x}}_{im} = \begin{bmatrix} x_{im} \\ y_{im} \\ 1 \end{bmatrix} = K \begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix}$$



INTRINSIC PARAMETERS²

In 2D (side view) this looks like this:

- Objects closer by appear larger

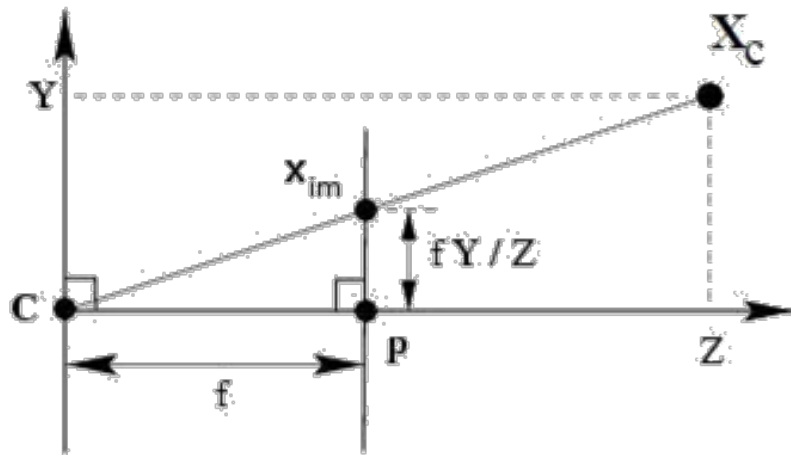


INTRINSIC PARAMETERS³

f is focal length, or distance to (virtual) image plane

Our aim is to find x_{im} (x_{im}, y_{im}, f), given X_c and f :

$$\begin{bmatrix} x_{im} \\ y_{im} \\ f \end{bmatrix} = \frac{f}{Z_c} \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix}$$



INTRINSIC PARAMETERS⁴

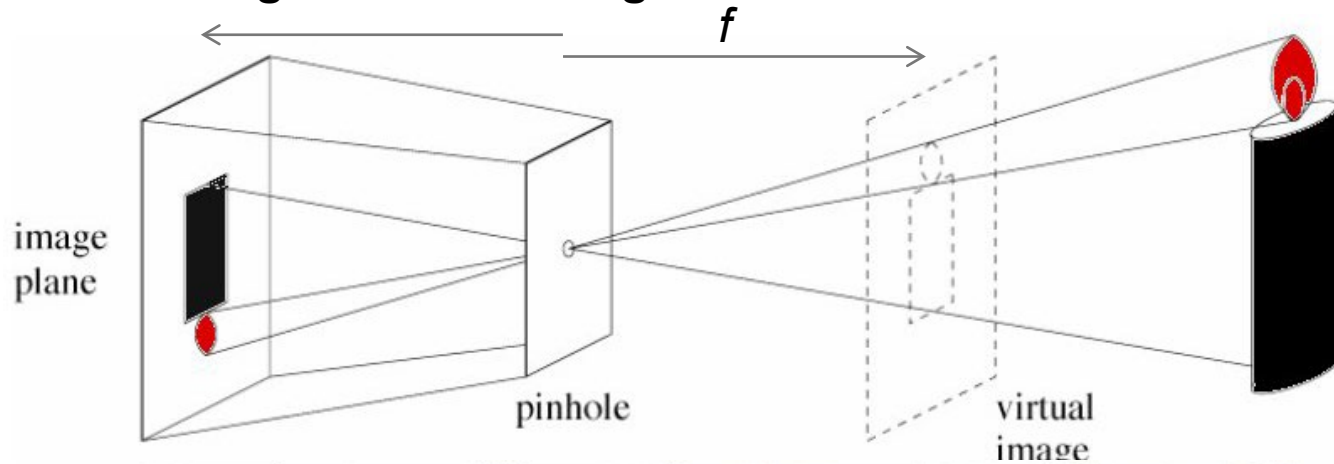
Instead of vector notation: $\begin{bmatrix} x_{im} \\ y_{im} \\ f \end{bmatrix} = \frac{f}{Z_c} \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix},$

we use matrix notation: $\begin{bmatrix} x_{im} \\ y_{im} \\ f \end{bmatrix} = \begin{bmatrix} f/Z_c & 0 & 0 \\ 0 & f/Z_c & 0 \\ 0 & 0 & f/Z_c \end{bmatrix} \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix}$

x_{im} is in pixels and so is f

INTRINSIC PARAMETERS⁵

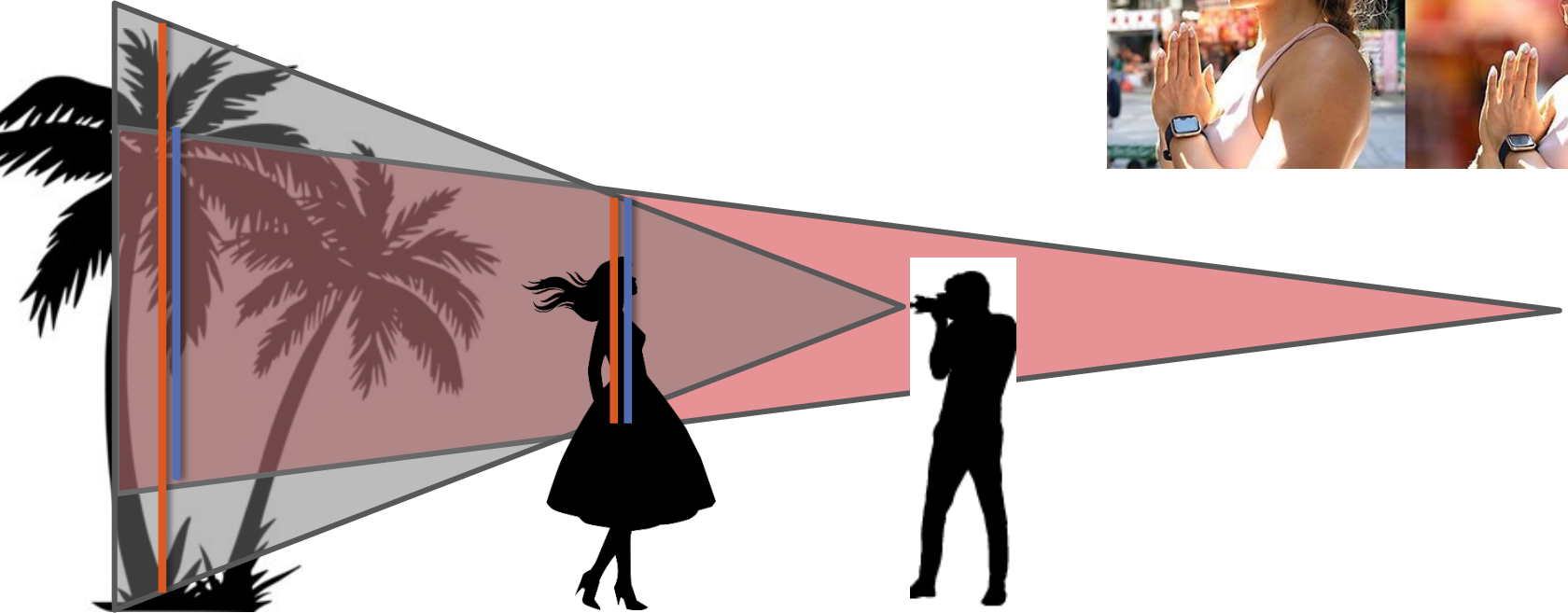
Effect of increasing and decreasing f





INTRINSIC PARAMETERS⁶

Effect of zooming

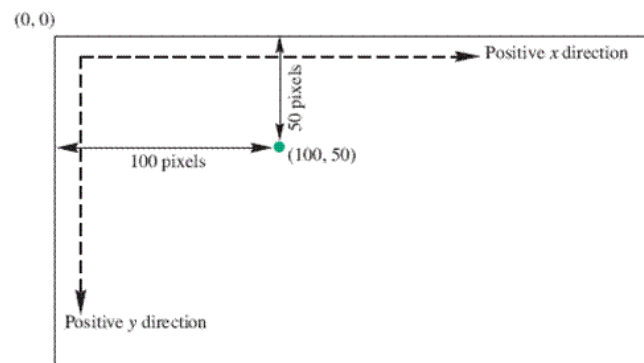
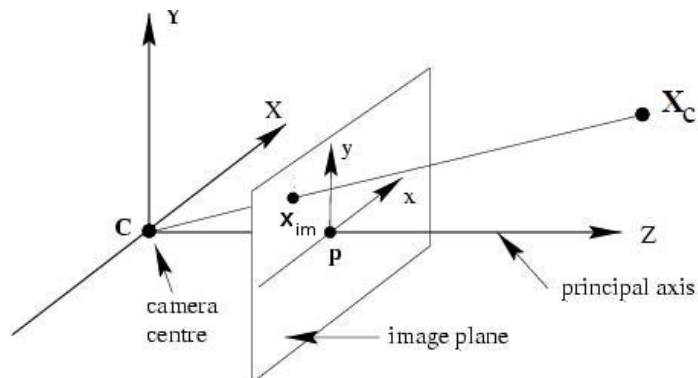


INTRINSIC PARAMETERS⁷

We can now project 3D to 2D, with the 2D origin in the center

- But we always number pixels from left-top to lower-right corner
- We need a translation

Not possible in matrix-form in current notation!



HOMOGENEOUS COORDINATES

Scaling and rotation can be expressed as matrix-vector multiplications

- Impossible with translation

Can be solved by using homogeneous coordinates:

- $\mathbf{x} = \begin{pmatrix} x \\ y \end{pmatrix}$ converts to $\tilde{\mathbf{x}} = \begin{pmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{w} \end{pmatrix} = \begin{pmatrix} \tilde{w}x \\ \tilde{w}y \\ \tilde{w} \end{pmatrix}$
- Additional scaling element $\tilde{w}$

HOMOGENEOUS COORDINATES²

Homogeneous coordinate $\tilde{x}$ is an alternative representation for x :

$$\tilde{x} = (\tilde{x} \quad \tilde{y} \quad \tilde{z} \quad \tilde{w}) = (\tilde{w}x \quad \tilde{w}y \quad \tilde{w}z \quad \tilde{w}) = \tilde{w}(x \quad y \quad z \quad 1)$$

$$x = (x \quad y \quad z) = (\tilde{x}/\tilde{w} \quad \tilde{y}/\tilde{w} \quad \tilde{z}/\tilde{w})$$

$$\bar{x} = (x \quad y \quad z \quad 1) = (\tilde{x}/\tilde{w} \quad \tilde{y}/\tilde{w} \quad \tilde{z}/\tilde{w} \quad \tilde{w}/\tilde{w}).$$

When $\tilde{w} = 0$, the point is at infinity

Conversion is straightforward for $\tilde{w} = 1$

HOMOGENEOUS COORDINATES³

Using homogeneous coordinates, we can express a translation as:

$$\bullet \begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} x + d_x \\ y + d_y \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & d_x \\ 0 & 1 & d_y \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

In inhomogeneous coordinates, it would multiply the translation

- Translation would depend on the coordinate
- In homogeneous coordinates, we add a fixed translation at a fixed scale (1)

HOMOGENEOUS COORDINATES³

Why homogeneous coordinates?

- Points at infinity can be expressed
- Transformations can be expressed in matrix form
- Perspective projections are straightforward

INTRINSIC PARAMETERS⁷

Recall: $x'_{im} = \begin{bmatrix} x_{im} \\ y_{im} \\ f \end{bmatrix} = \begin{bmatrix} f/Zc & 0 & 0 \\ 0 & f/Zc & 0 \\ 0 & 0 & f/Zc \end{bmatrix} \mathbf{X}_c$

The projection in homogeneous coordinates: $\tilde{x}'_{im} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \tilde{\mathbf{X}}_c$

We easily add translation (x_0, y_0) in the image plane: $\tilde{x}'_{im} = \begin{bmatrix} f & 0 & x_0 & 0 \\ 0 & f & y_0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \tilde{\mathbf{X}}_c$

INTRINSIC PARAMETERS⁸

$\begin{bmatrix} f & 0 & x_0 \\ 0 & f & y_0 \\ 0 & 0 & 1 \end{bmatrix}$ is the intrinsic camera matrix K

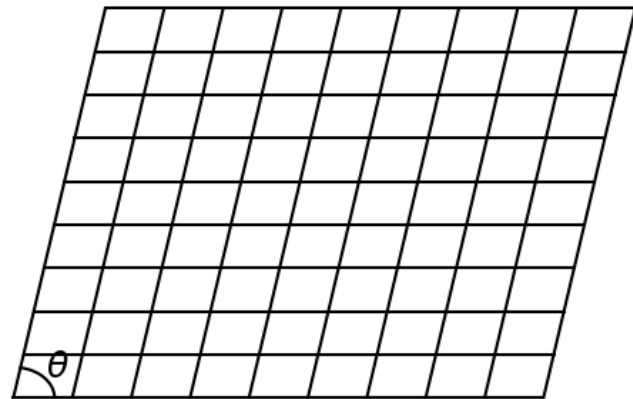
- It projects (based on focus length) and translates (based on principal point)
- Does not model properties of the lens or sensor apart from the focal length

INTRINSIC PARAMETERS⁹

Typically, we use a generalized version of K :
$$\begin{bmatrix} f_x & s & x_0 \\ 0 & f_y & y_0 \\ 0 & 0 & 1 \end{bmatrix}$$

With skew coefficient $s = \tan \theta$, θ the angle between the image axes

- If $s=0$ and $f_x=f_y$, then pixels are perfectly square
- If $s=0$ and $f_x \neq f_y$, then pixels are rectangular
- If $s \neq 0$ and $f_x=f_y$, then pixels are skewed

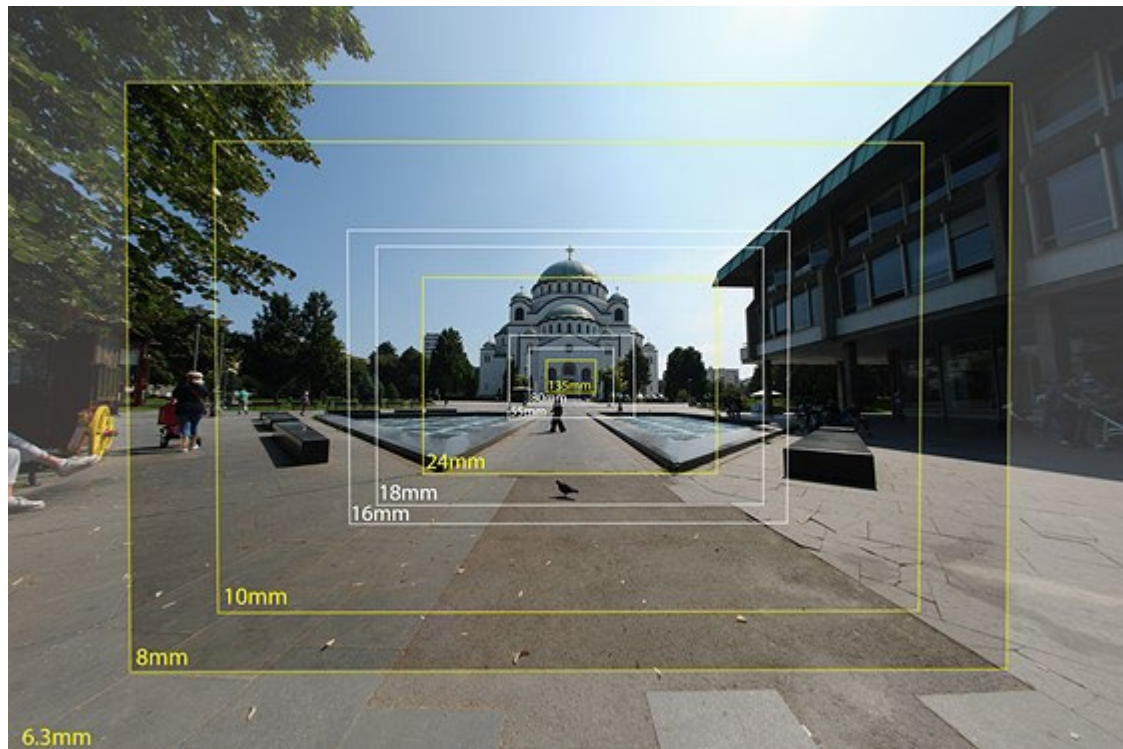


INTRINSIC PARAMETERS¹⁰

$$K: \begin{bmatrix} f_x & s & x_0 \\ 0 & f_y & y_0 \\ 0 & 0 & 1 \end{bmatrix}$$

Q: Which parameters change when we zoom in?

- Only focal length



INTRINSIC PARAMETERS¹¹

$$\mathbf{K}: \begin{bmatrix} f_x & s & x_0 \\ 0 & f_y & y_0 \\ 0 & 0 & 1 \end{bmatrix}$$

Q: Which parameters change when we increase the resolution?

- Focal length in pixels $\rightarrow$ increases
- Image origin in pixels $\rightarrow$ increases

RADIAL DISTORTION

With perspective projection, straight lines remain straight after projection

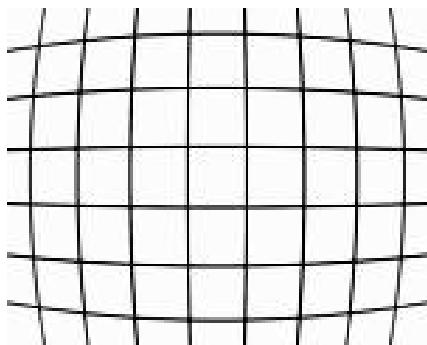
- Not the case when using wide-angle lenses
- These lenses cause radial distortion



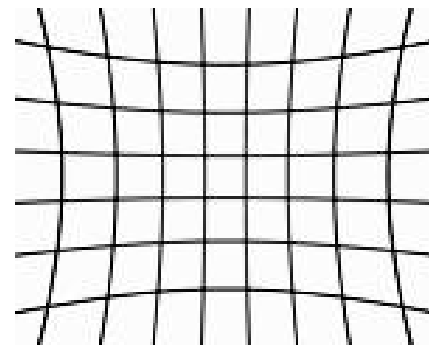
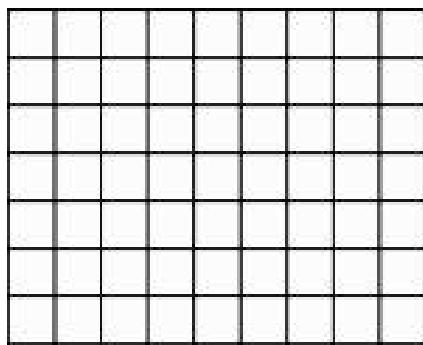
RADIAL DISTORTION²

Radial distortion can be corrected

- Additional parameters to be estimated during calibration
- Can be done using OpenCV functions, but not discussed in the lectures



Barrel distortion
(Fish-eye lens)

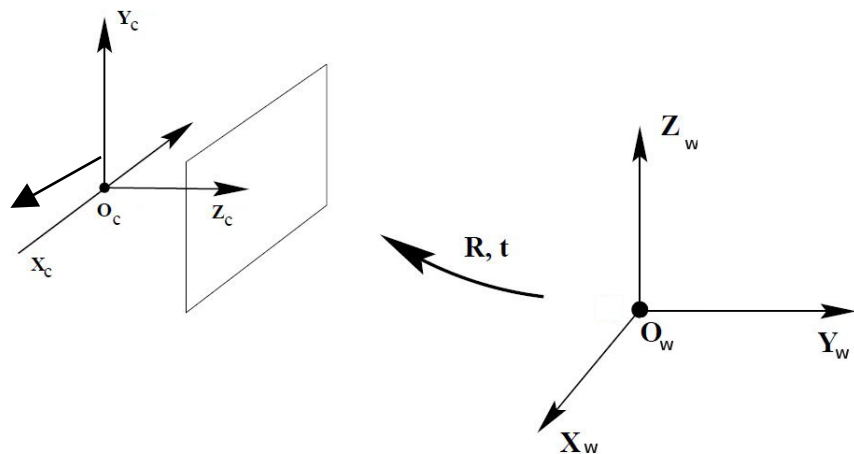


Pincushion distortion

CAMERA MATRIX REVISITED

Perspective projection consists of:

- Projection from 3D world to 3D camera coordinates: extrinsic
- Projection from 3D camera to 2D image coordinates: intrinsic



CAMERA MATRIX REVISITED²

Extrinsic parameters (project from world to 3D camera coordinates):

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = [R \quad t] \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}$$

Intrinsic parameters (project from 3D camera to image coordinates):

$$\begin{bmatrix} x_{im} \\ y_{im} \\ 1 \end{bmatrix} = K \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix}$$

CAMERA MATRIX REVISITED³

Camera matrix P combines both:

- $P = K[R \ t]$

K is a 3 x 3 matrix, $[R \ t]$ a 3 x 4 matrix

- P is a 3 x 4 matrix
- Typically left as $K[R \ t]$

CAMERA MATRIX REVISITED⁴

P projects 3D world coordinates onto 2D image coordinates:

$$\bar{x}_{im} = P\bar{X}_{world}$$

$$P = K[R \quad t]$$

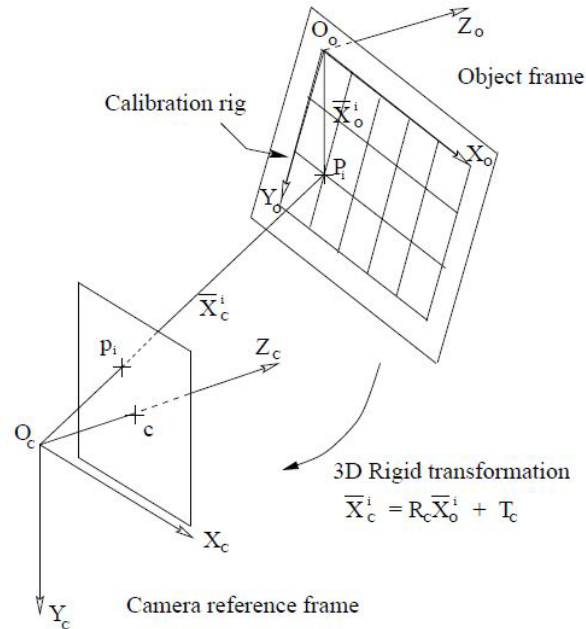
The aim of geometric calibration is to find parameters in K , R and t :

- Calculate K only once for a fixed camera setting (i.e., zoom)
- Re-calculate R and t everytime we move the camera

CALIBRATION

CALIBRATION

Aim of calibration is to determine the extrinsic and intrinsic parameters from images of an object with known properties



CALIBRATION²

Typically, a chessboard or checkerboard is used:

- Contrast between dark and light
- Distance between crossings is equal
- All crossings are in the same plane
- Large number of crossings visible

Convention: left upper corner of the first image is taken as world origin

- x- and y-axes along the sides of the checkerboard
- z-axis is orthogonal, so points out of the chessboard

CALIBRATION³

Basic idea:

Set of 3D
points

(crossings of the checkerboard, which we assume is in a fixed plane, with known 3D coordinates)

Unknown projection parameters



Corresponding
set of 2D
points

(e.g.,
chessboard
corner points
found in the
image)

CALIBRATION⁴

We need to estimate multiple parameters in K , R and t :

- Paired sets of 2D points and 3D points

Once we know the real distance between crossings, we obtain a series of equations with some unknowns:

- The unknowns are the parameters of our camera matrix

The number of equations increases with the measurements:

- Solved by iterative minimization of error

CALIBRATION⁵

Estimating the parameters from a single image is not smart:

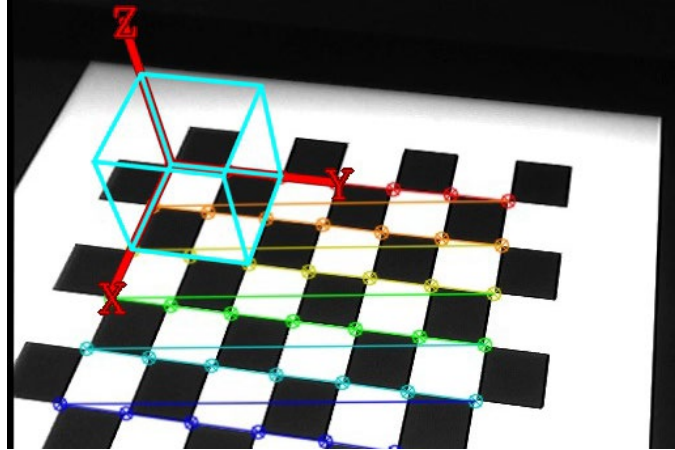
- Robustness is low as there is inaccuracy in the localization of the crossings
- More images $\rightarrow$ more equations $\rightarrow$ more robust calibration



CALIBRATION⁶

Intrinsic and extrinsic parameters are estimated simultaneously

- There is no (easy) way to measure 3D camera coordinates



BREAK!

RADIOMETRY

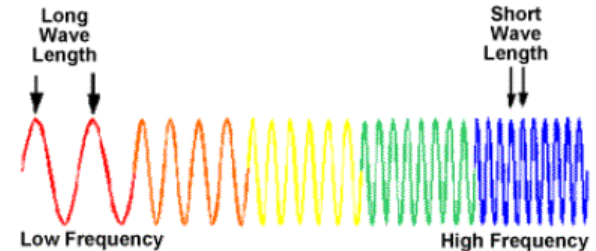
FROM LIGHT TO IMAGE

Digital cameras convert light energy to electric charges

- Light has an intensity (amplitude) and a wavelength
- Intensity determines the strength of the charge
- Wavelength determines the color

Digital cameras use a grid of light-absorbing sensors

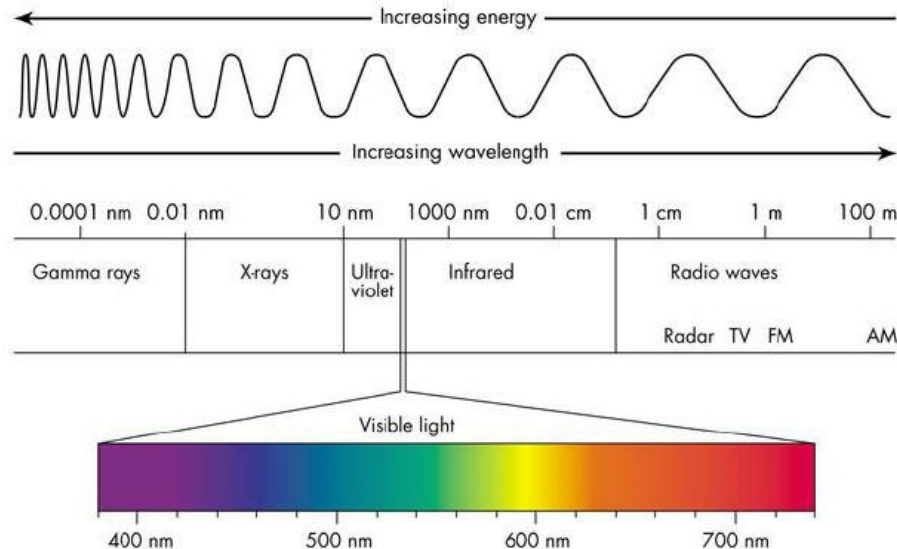
- Electric charges are converted to “intensity” values
- Each sensor is sensitive to range of wavelengths



WAVELENGTH

Humans are sensitive to wavelengths of 380-740nm

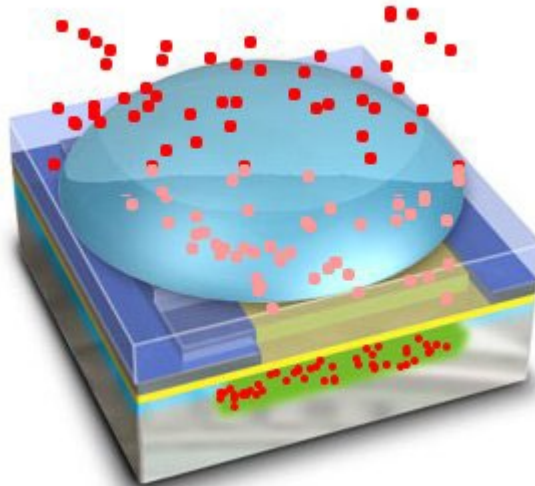
- Cameras are typically tuned to this range as well
- Near-infrared (Kinect) or infrared (thermal) sometimes used



RECORDING LIGHT

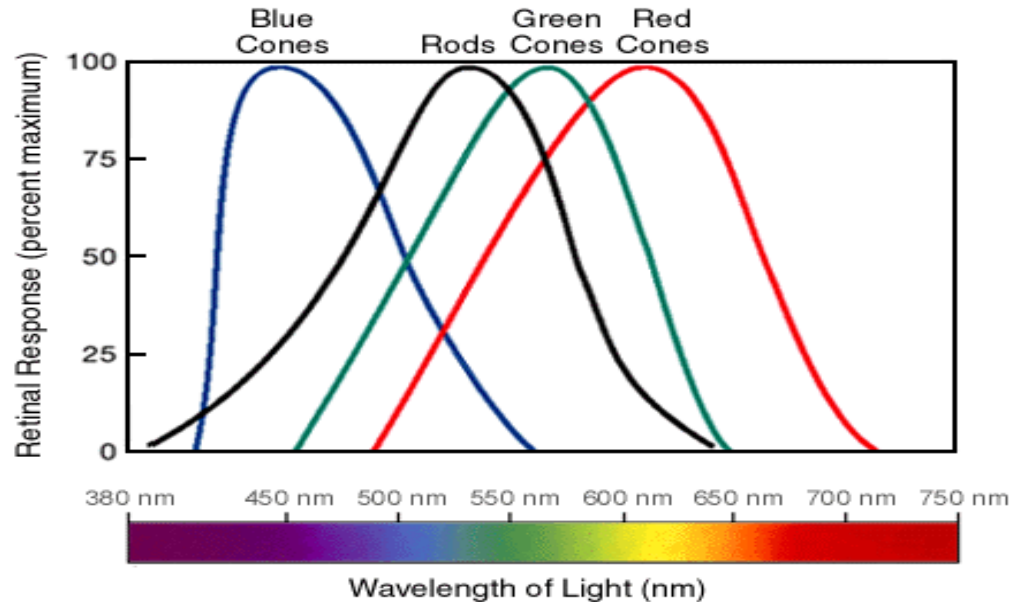
Cameras use CMOS or CCD, grids of imaging sensors

- Convert light energy to electric charges
- Is mono-chromatic, only intensity taken into account



RECORDING COLOR

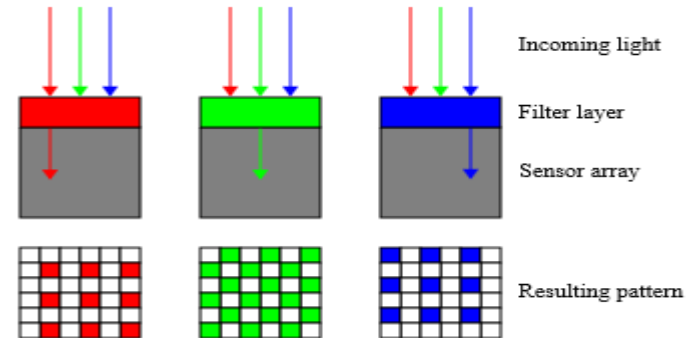
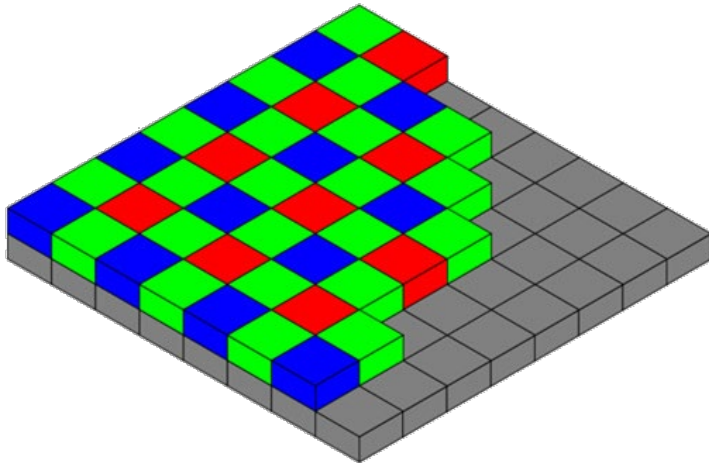
To perceive color, CCD works just like cones in the human eye



RECORDING COLOR²

To record color, three different sensors are used

- Each sensitive to another range of colors
- Color is determined as function of sensor values in a region
- Human eye is more sensitive to luminance, mostly determined by green



RECORDING COLOR³

Outputs eventually transformed to discrete pixel values

- Different cameras have different color function
- Automatic white-balance control can be achieved
- Post-processing can be used (noise reduction, contrast)



ASSIGNMENT WALKTHROUGH

ASSIGNMENT WALKTHROUGH

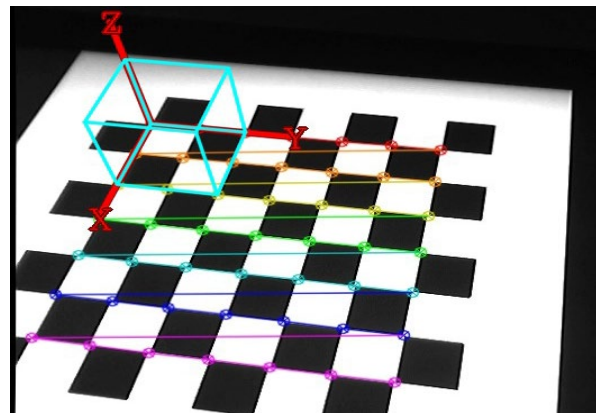
Geometric calibration using OpenCV:

- Introduction to OpenCV en Python
- Needed for Assignment 2
- OpenCV functions: https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html

Focus on the intrinsic camera parameters!

- The extrinsics change in each frame
- Used only for coloring

“Offline” and an “online” part in the assignment



ASSIGNMENT WALKTHROUGH²

Workflow offline:

- Print checkerboard on piece of paper (or take a real one, or use a tablet)
- Fix it on an object that cannot bend (!)
- Measure the stride length and fill it in at the proper place in your code
- Take a number of pictures of your checkerboard with the camera
- Determine camera parameters using OpenCV functions (browse!)
- Now your camera is calibrated

Workflow online:

- Read an image/camera frame
- Find corner points and estimate camera extrinsics (browse functions!)
- Draw a box on a detected chessboard in the right perspective

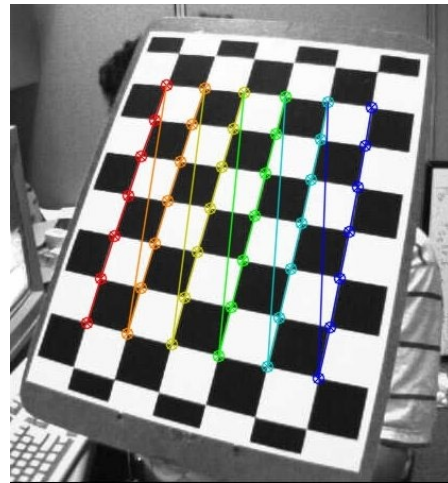
ASSIGNMENT WALKTHROUGH³

There is a function that can find the chessboard corners for you

- Doesn't always produce an output

Implement a function to manually set the four corner points and interpolate the chessboard tile corners

- Not ideal but that's okay
- Choice points for improving the interpolated positions

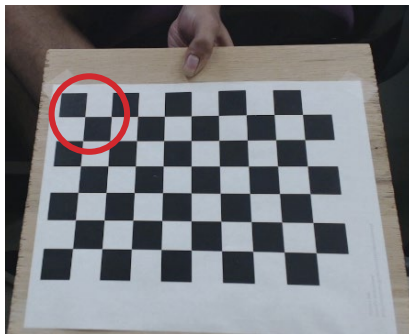


ASSIGNMENT WALKTHROUGH⁴

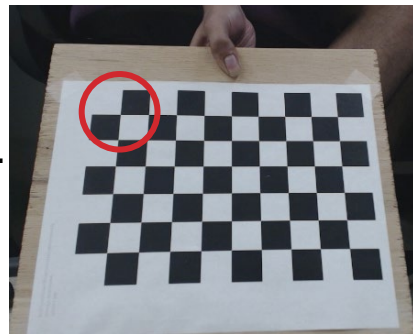
Calibration quality depends on the variation in position of the chessboard

- Make sure you cover the borders and corners of the view well
- Tilt/rotate the chessboard
- Minimum number of images is specified

Make sure the same corner is always left top



vs.



ASSIGNMENT WALKTHROUGH⁵

You can check which photos are not processed correctly

- Meaning: no corners are found
- Keep at least 5 of these images in your “training” set

Typically, the chance of correct detection is lower when:

- The resolution is lower
- There are reflections on the chessboard
- There is motion blur

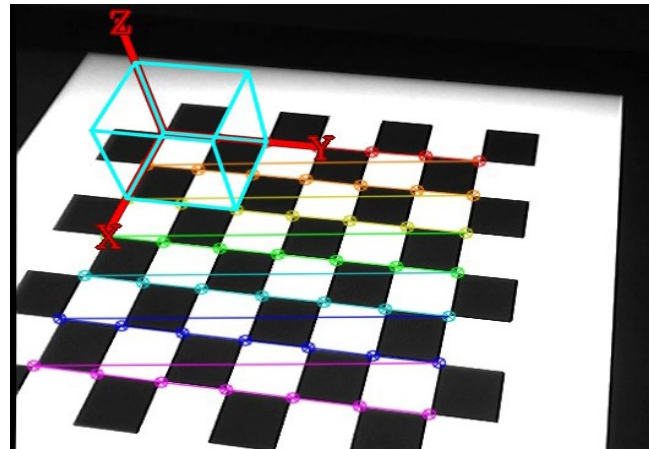
You can artificially blur your images if the corner detection never fails

ASSIGNMENT WALKTHROUGH⁶

Based on one test image, draw a cube on your chessboard. You can **ONLY** use (projected) 3D information for the drawing

The top area of the box should be colored according to the relative position and orientation of the chessboard

- Can be found in estimated extrinsics



COMMON MISTAKES

Don't fix the horizontal and vertical focal length

Don't specify (rather than calculate) the center/origin

- Solution: check which flags are available

Bad calibration, occasional flipping of the direction of the z-axis

- Check if chessboard is not rotated

Use 2D (not 3D) information to color the cube

CHOICE TASKS

CHOICE 1: real-time performance with webcam (max 10 points)

- Use frames in realtime from your webcam in the online phase
- Move the chessboard closer and further from the camera, and rotate it
- Add a link to a video in your report

CHOICE TASKS²

CHOICE 2: implement a function to remove inaccurate inputs

- In general: the more images you have, the better your calibration
- So, leaving out an image should decrease the calibration quality
- If this is not the case, keep that image out

How do you measure the quality of the calibration?

How do you know if an additional image doesn't improve the results?

- Check for outputs of calibration functions that might be used

CHOICE TASKS³

CHOICE 3: improving the localization of the corner points in your manual interface: 10. Make the estimation of (a) the interpolated points, or (b) the four corner points more accurate.

Linear interpolation will have a strong effect on the localization when there are strong perspective effects

- Use image content (e.g., locations of strong contrast) to improve the estimation of all points (option a). Use of cornerSubPix is not sufficient, use warping back and forth.
- Alternatively, improve the localization of the four corner points after clicking (option b, max 5 points)

CHOICE TASKS⁴

CHOICE 4: provide the confidence for how well each variable in your intrinsics matrix has been estimated: 10

Use different subsets to estimate the confidence of each variable (focus length, origin).

CHOICE TASKS⁵

CHOICE 5: enhance the input to reduce the number of input images that are not correctly processed by findChessboardCorners, for example by enhancing edges or getting rid of light reflections: 10

Implement a preprocessing step

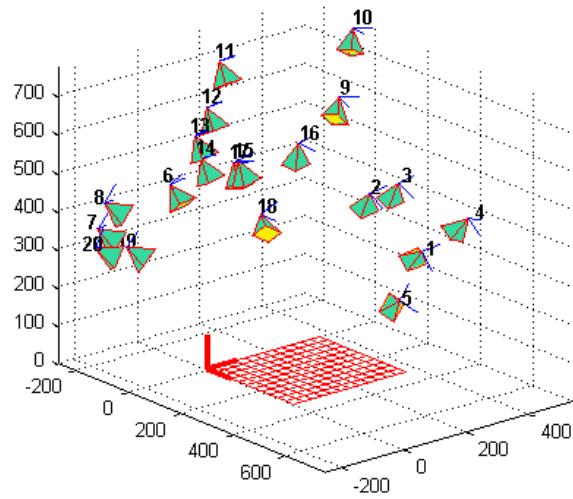
- In your report, mention the effect of your preprocessing on the calibration quality (number of discarded images, change in reprojection error)
- Report the results with and without enhancement

CHOICE TASKS⁶

CHOICE 6: produce a 3D plot with the locations of the camera relative to the chessboard when each of the training images was taken: 10

For each training image, obtain the 3D location of the camera relative to the chessboard

- Excellent opportunity to explore matplotlib



CHOICE TASKS⁷

The total points for choice tasks for this assignment is 30

- If you (partially) fail one task, no points will be subtracted
- Mention all choice tasks you performed

If you have other ideas, ask me about the suitability and grading

ASSIGNMENT WALKTHROUGH⁹

Deadline Wednesday February 18, 23:00

- In pairs. Can't find a partner: use Teams or send me a message
- If you want to work alone: send me a message
- Submission in Brightspace

Use Teams to ask questions

- Next: Assignment support session (15:15-17:00) in BBG-214/219

FINALLY...

NEXT LECTURE

3D computer vision

- Using camera geometry to estimate 3D structures
- Focus on one algorithm: silhouette-based volume reconstruction
- Basis for Assignment 2, requires Assignment 1

Thursday February 12, 11:00-12:45, RUPPERT-Paars

MATERIALS

Computer Vision: Algorithms and Applications (Szeliski) 2, 10.1 and 11.1-11.2

Background materials:

- [How cameras work](#)
- [How digital cameras work](#)
- [Homogenous Coordinates](#)
- [Camera matrix](#)
- [Introduction to light and color](#)